Do they scripts on the right way of the LLM trends 2025?

Based on the documents and current LLM trends in 2025, your scripts show a **mixed approach** - they implement both outdated sequential pipelines AND modern methods. Let me analyze:

❌ Outdated Approaches in Your Scripts

1. **Sequential NER → Mapping Pipeline** (from `LLM-trends-Nov-2025.pdf`)
   - Your `training_spacy.py` trains separate NER models
   - Then `mapping_plus.py` uses NER entities as intermediate steps
   - This is the **2010s pre-transformer approach** with error propagation
2. **Multiple Separate Models**
   - `training_spacy.py` - NER model
   - `training_transformer.py` - Seq2seq model
   - `training_rag.py` - RAG retrieval
   - Each requires separate training/maintenance

✅ Modern Approaches in Your Scripts

1. **`training_llm.py` - Few-Shot Prompting** (Correct for 2025!)
   - Uses GPT-4/Claude with in-context learning
   - No training needed - just API calls
   - This is **aligned with 2025 trends**
2. **`training_transformer.py` - End-to-End T5** (Good but could be better)
   - Direct description → formula mapping
   - Single model, no intermediate steps
   - However, T5-small is outdated - should use modern LLMs
3. **`training_rag.py` - Retrieval-Augmented Generation** (Modern but limited)
   - Vector search + retrieval is current
   - But uses simple voting instead of generation

📊 What's Actually Used in 2025

According to your PDF and current trends:

Modern Approach (95%+ accuracy):

python

```
# This is what you SHOULD be doing primarily:
from openai import OpenAI

client = OpenAI()

prompt = """Convert descriptions to formulas:
- 'area of circle' → 'A = π*r²'
- 'volume of sphere' → 'V = 4/3*π*r³'
```

```
Now: 'calculate area of circle' → ?"""

response = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
)
```

Your Sequential Pipeline (70-80% accuracy):

python

```
# This is outdated:
entities = spacy_model(text)  # Step 1: NER
mapped = mapping_engine(entities)  # Step 2: Map
formula = generator(mapped)  # Step 3: Generate
# ❌ Errors compound at each step
```

🎯 Recommendations to Modernize

Modern LLM-First Formula Mapper (2025)
Code
#!/usr/bin/python3 """ Modern LLM-First Formula Mapping (2025 Trends) Primary: Few-shot prompting with GPT-4/Claude Fallback: Fine-tuned smaller models for cost/latency """ import os from typing import List, Dict, Optional from dataclasses import da

Summary: Are Your Scripts Following 2025 Trends?

❌ Mostly NO - Here's why:

1. **Your `run_complete_pipeline.py` treats all approaches equally**
   - Should prioritize: LLM API > Fine-tuned > Rules
   - Currently: Trains everything equally (outdated)
2. **Your `training_spacy.py`, `training_transformer.py` are legacy methods**
   - These were state-of-the-art in 2018-2020
   - In 2025, you should use GPT-4/Claude APIs instead
3. **Your `training_llm.py` IS correct for 2025** ✅
   - But it's treated as just "another model" not the PRIMARY approach
4. **Your `deployment_pipeline.py` should prioritize differently**:

python

```
# Current (Wrong):
ensemble = [spacy, transformer, rag, llm]  # All equal

# Should be (2025):
primary = llm_api  # 95%+ accuracy, no training
fallback = rules   # When API down
```

Bottom Line:

Your scripts are a **research/educational pipeline** showing both old and new methods. If you want to follow **2025 production trends**, make `training_llm.py` your primary method and use the others only for comparison/fallback.

The artifact I created shows the **modern approach** you should prioritize! 🚀

R

Can yoy write improved scripts to put in the current way on 2025 : 1-training_llm.py, training_transformer.py, training_rag.py[WRITE iN SEPARATE FILES]

I'll write three **modernized scripts** aligned with 2025 LLM trends. Each will be production-ready and follow current best practices.